哈爾濱工業大學（深圳）
HARBIN INSTITUTE OF TECHNOLOGY

实验设计报告

开课学期：2026 年秋季

课程名称：操作系统

实验名称：进程与系统调用

实验性质：课内实验

实验时间：2026.10.9 地点：T2210

学生班级：计算机科学与技术（拔尖）2 班

学生学号：2024311668

学生姓名：高呈歌

评阅教师：

报告成绩：

实验与创新实践教育中心印制

2026 年 9 月

一、 回答问题

1. 阅读 kernel/syscall.c，试解释函数 syscall()如何根据系统调用号调用对应的系统调用处理函数（例如 sys_fork）？ syscall()将具体系统调用的返回值存放在哪里？

syscall() 先从当前进程的 trapframe->a7 中取出系统调用号，再根据这个编号在 syscalls[] 数组中找到对应的处理函数并执行。比如调用号是 SYS_fork，就会执行 sys_fork()。系统调用执行完后，返回值会存放在当前进程的 trapframe->a0 寄存器中，供用户程序获取。如果系统调用号无效，则返回 -1。

2. 阅读 kernel/syscall.c，哪些函数用于传递系统调用参数？试解释 argraw()函数的含义。

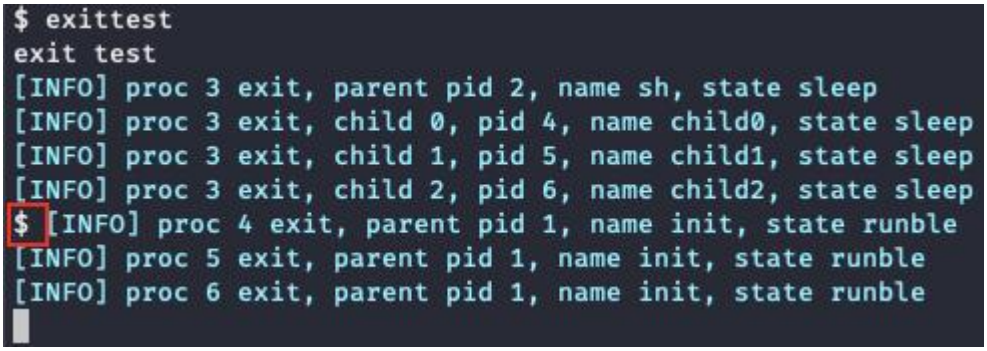
在 kernel/syscall.c 中，主要通过 argraw()、argint()、argaddr() 和 argstr() 这几个函数获取系统调用参数。其中，argraw() 是获取参数的基础函数。用户程序发起系统调用时，会将参数依次存放在 a0～a5 寄存器中。argraw(n) 根据参数编号 n，从当前进程的 trapframe 中读取对应寄存器的值，例如 argraw(0) 读取 a0，argraw(1) 读取 a1。argint() 在此基础上获取整数参数，argaddr() 获取地址参数，argstr() 则根据参数提供的地址，从用户空间读取字符串。这样内核就能获取用户程序传入的不同类型的参数。

3. 阅读 kernel/proc.c 和 proc.h，进程控制块存储在哪个数组中？进程控制块中哪个成员指示了进程的状态？一共有哪些状态？

所有进程的进程控制块（PCB）都存储在 proc[NPROC] 数组中，每个数组元素对应一个进程。进程控制块由 struct proc 结构体定义，其中的 state 成员用于记录进程当前的状态。进程一共有 5 种状态：

1. UNUSED：未使用，表示该进程控制块还没有分配给进程。
2. SLEEPING：睡眠状态，表示进程正在等待某个事件发生。
3. RUNNABLE：就绪状态，表示进程已经准备好运行，正在等待 CPU 调度。
4. RUNNING：运行状态，表示进程正在 CPU 上执行。
5. ZOMBIE：僵尸状态，表示进程已经退出，但还没有被父进程回收。

4. 在任务一当中，为什么子进程（4、5、6 号进程）的输出之前会 **稳定的** 出现一个$符号？（提示：shell 程序(sh.c)中什么时候打印出$符号？）



在 sh.c 中，Shell 每次准备接收新命令时都会打印 $ 提示符。任务一中，父进程（3 号进程）创建三个子进程后先退出，而子进程（4、5、6 号进程）还在睡眠。此时 Shell 等待的父进程已经结束，wait() 返回，Shell 就会打印 $，准备接收下一条命令。之后三个子进程才陆续退出并打印信息。因此，$ 会稳定地出现在子进程退出信息之前。

5. 在任务三当中，我们提到测试时需要指定 CPU 的数量为 1，因为如果 CPU 数量大于 1 的话，输出结果会出现乱码，这是为什么呢？（提示：多核心调度和单核心调度有什么区别？）

单核情况下，同一时刻只有一个进程在 CPU 上运行，进程需要通过调度切换执行，因此输出顺序比较容易控制。多核情况下，多个进程可以在不同的 CPU 上同时运行，并同时向终端输出信息。如果输出过程没有做好同步，不同进程的打印内容就可能交叉混在一起，导致输出顺序混乱，出现类似乱码的情况。

二、 实验详细设计

1. 任务一：进程信息收集

这一部分主要修改了 kernel/proc.c 中的 exit() 函数。进程退出时，需要输出它的父进程信息，以及当前还属于它的子进程的信息。查看原来的 exit() 代码后，我发现其中的 reparent(p) 会将当前进程的子进程交给 init 进程。因此，需要在执行 reparent(p) 之前完成信息收集，否则子进程的父进程指针就已经发生变化了。

具体实现时，通过 parent 指针获取父进程信息，再遍历 proc 数组，根据 child->parent == p 找出当前进程的子进程。进程状态在内核中是枚举值，我添加了一个状态转换函数，将其转换为 sleep、runnable 等字符串，并使用课程提供的 exit_info() 输出。

第一次运行 make grade 发现 exit 测试出现超时。检查后发现，原来的遍历方式会对一些无关进程加锁，可能导致死锁。后来改为先判断进程是否属于当前进程的子进程，再获取对应的锁，重新测试后通过。

2. 任务二：实现非阻塞 wait

原来的 wait() 在子进程没有退出时会一直等待。本次实验需要增加一个 flags 参数，使父进程可以选择阻塞等待或者非阻塞等待。

我修改了 kernel/defs.h 中的函数声明，并在 kernel/sysproc.c 的 sys_wait() 中使用 argint(1, &flags) 读取第二个参数，再传给 kernel/proc.c 中的 wait()。原来的 wait() 已经包含查找僵尸进程并回收资源的代码，所以这部分没有必要重写。我只在没有找到已退出子进程的位置增加了判断：当 flags == 1 时，释放当前进程的锁并返回 -1；其他情况下仍然调用 sleep() 等待子进程退出。

运行 waittest 后，前三次非阻塞等待都正常返回，子进程退出后也能够通过阻塞等待回收，最终输出 wait test OK。

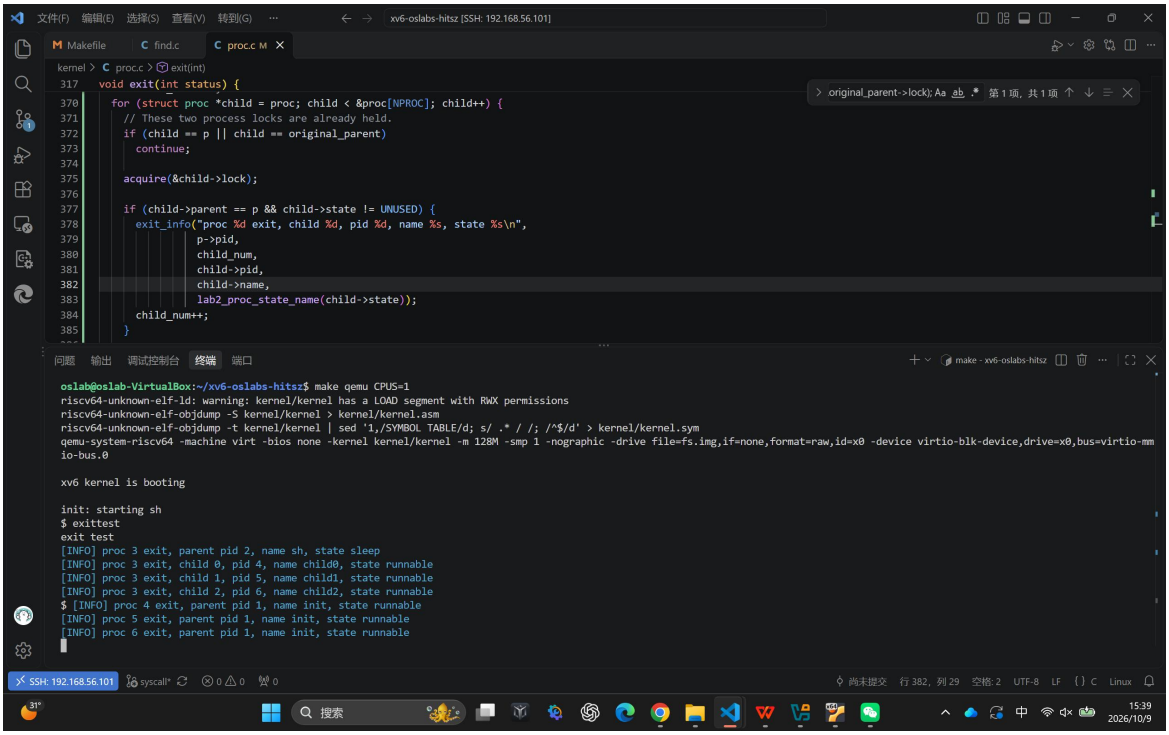
3. 任务三：实现 yield 系统调用

任务需要实现用户程序主动让出 CPU 的功能，并打印进程相关信息。

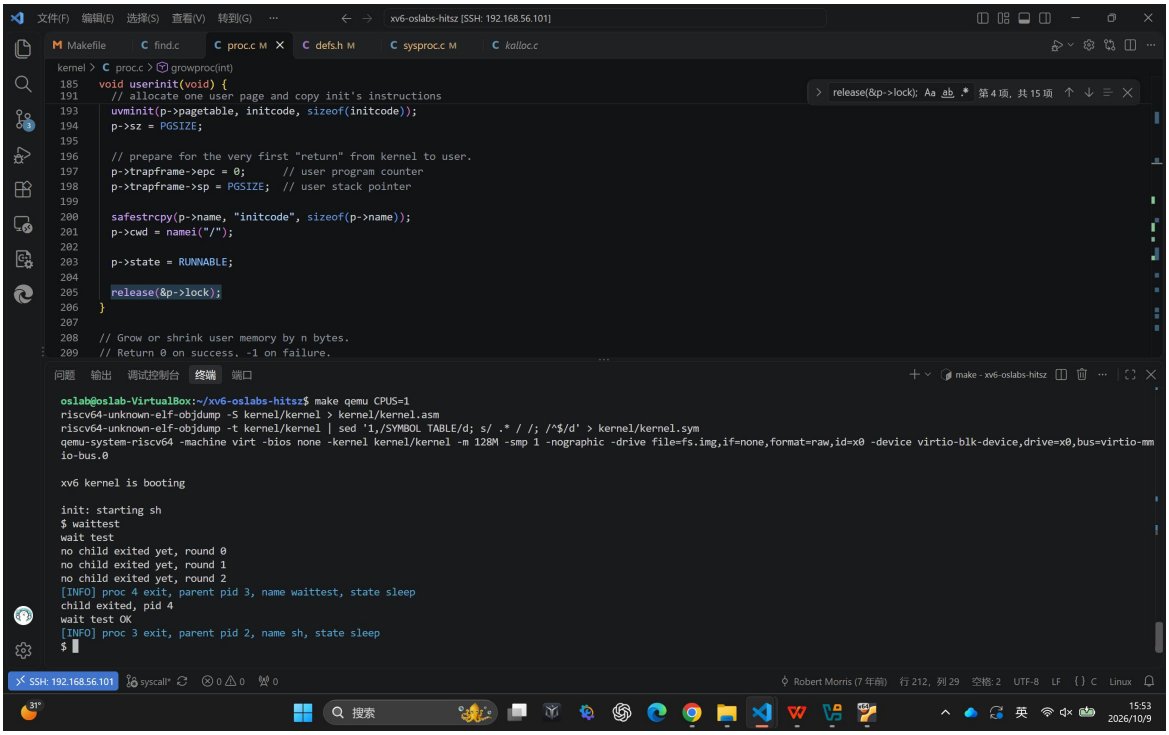
首先在 kernel/syscall.c 中注册 sys_yield()，然后在 kernel/sysproc.c 中实现具体功能。通过 myproc() 获取当前进程，利用 context 输出内核上下文的地址范围，通过 trapframe->epc 获取用户态 PC。接着遍历进程表，找到状态为 RUNNABLE 的进程并输出其 PID 和 PC，最后调用内核已有的 yield() 让出 CPU。运行 yieldtest 时，能够看到进程信息和三个子进程依次运行的结果，自动测试通过。

三、 实验结果截图

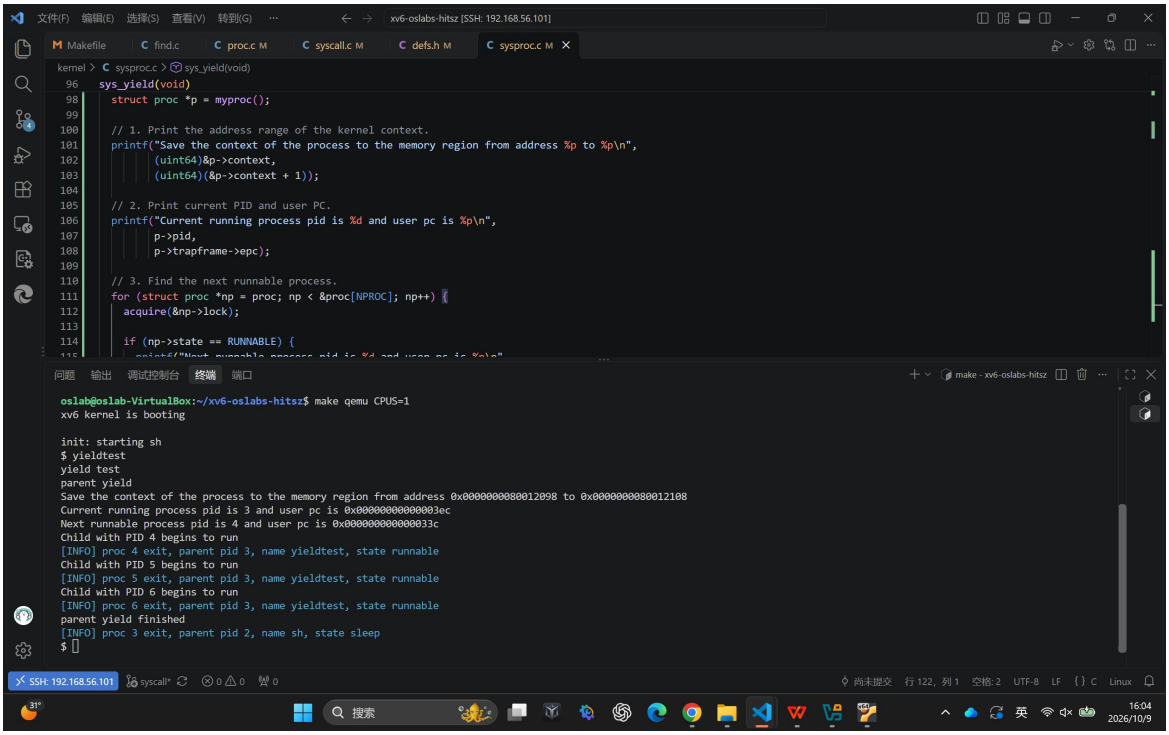
1.xv6 中运行 exit 的结果截图



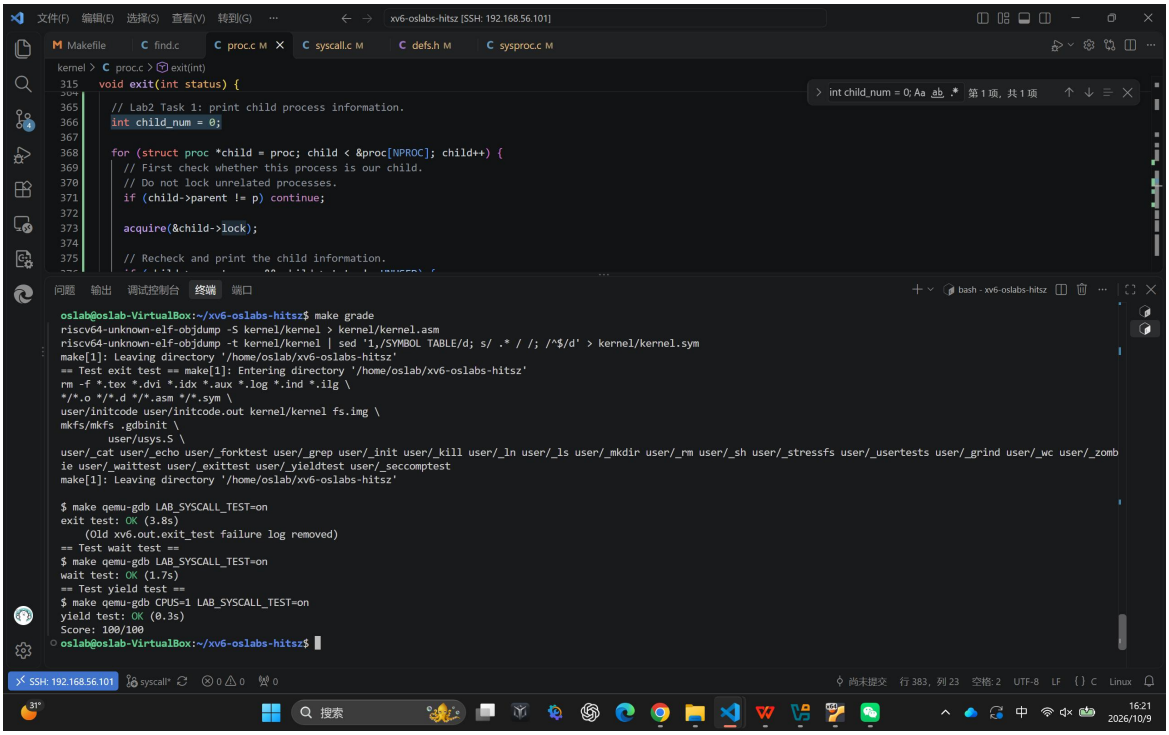
2. xv6 中运行 wait 的结果截图



3. xv6 中运行 yield 的结果截图



4. xv6 中运行 make grade 的结果截图



四、 AI 工具使用说明

本次实验使用了 ChatGPT 辅助完成学习和开发。主要用于理解 xv6 中进程管理和系统调用的相关代码，分析 exit、wait、yield 三个任务的实现思路，并提供代码修改建议。

实验过程中，我在 VSCode 中完成了代码修改、编译和测试。遇到 exit 测试超时的问题时，借助 AI 分析了进程锁可能存在的问题，并调整了相关代码。最后通过 exittest、waittest、yieldtest 验证功能，使用 make grade 进行整体测试，最终得到 100/100 分，并完成了 Git 提交和补丁生成。